

MKPDesigns

Advanced architecture portfolio and project management platform built with the MERN stack.

Table of Contents

- [Features](#)
- [Tech Stack](#)
- [Data Coverage](#)
- [Project Structure](#)
- [Installation](#)
- [Environment Setup](#)
- [Running the Application](#)
- [API Documentation](#)
- [Dashboard Pages](#)
- [Data Sources](#)
- [Deployment](#)
- [Troubleshooting](#)
- [Contributing](#)
- [License](#)

Features

- **Authentication System**
 - Secure Email/Password Signup & Login
 - Google OAuth Integration
 - **Forgot Password:** Secure email-based password reset flow (OTP & Direct Link).
 - OTP (One-Time Password) Email Verification
 - Profile Completion Flow for Social Logins
- **Dashboard & Management**
 - **Projects:** Comprehensive project management view with details.
 - **Designs:** Architecture design portfolio with gallery view.
 - **Secure Downloads:**
 - Public resources available to all.
 - Private project details (CAD/PDF) restricted to authenticated purchasers.
 - **Payments & Invoicing:**

- Integrated **Razorpay** payment gateway for design purchases.
- Automated **PDF Invoice Generation** emailed to users and admins.
- Downloadable invoices from transaction history.
- **Admin Tools:**
 - Upload and manage Projects and Designs.
 - **Deletion Control:** Admins can safely delete projects and designs (auto-removes assets from Cloudinary).

// ... (rest of the file) ...

Server (server/.env):

```
PORT=5000

MONGO_URI=your_mongodb_connection_string

JWT_SECRET=your_jwt_secret

JWT_EXPIRE=30d

# Cloudinary

CLOUDINARY_CLOUD_NAME=your_cloud_name

CLOUDINARY_API_KEY=your_api_key

CLOUDINARY_API_SECRET=your_api_secret

# Email (Nodemailer)

EMAIL_SERVICE=gmail

EMAIL_USERNAME=your_email@gmail.com

EMAIL_PASSWORD=your_app_password

# Google Auth

GOOGLE_CLIENT_ID=your_google_client_id

GOOGLE_CLIENT_SECRET=your_google_client_secret

CLIENT_URL=http://localhost:5173

# Razorpay

RAZORPAY_KEY_ID=your_razorpay_key_id

RAZORPAY_KEY_SECRET=your_razorpay_key_secret

# Admin

ADMIN_EMAIL=your_admin_email@gmail.com
```

- **User Profile**

- Profile management with organizational details.
- Customizable preferences.
- **Security**
 - **Headers:** Implementation of Helmet for secure HTTP headers.
 - **Rate Limiting:** Brute-force protection on all routes (max 100 requests/15min per IP).
 - **Input Validation:** Strict server-side validation using Joi schemas.
 - **Sanitization:** Protection against XSS and HTTP Parameter Pollution.
- **Modern UI/UX**
 - Responsive design using Tailwind CSS.
 - **Interactive Elements:** Glassmorphism cards, animated backgrounds, and intuitive navigation.
 - Interactive 3D elements (Three.js/Fiber).
 - Dark/Light mode support.
 - Toast notifications for user feedback.

Tech Stack

For a detailed breakdown of libraries and their usage, see [LIBRARY_USAGE.md](#).

Client-Side

- **Core:** React 19, Vite
- **Styling:** Tailwind CSS 4, Lucide React (Icons), Framer Motion
- **State/Routing:** React Router 7, Context API
- **Visuals:** React Three Fiber (3D), React Image Gallery
- **Forms:** React Hook Form, Yup Validation
- **Auth:** React OAuth Google, JWT Decode

Server-Side

- **Runtime:** Node.js, Express.js
- **Database:** MongoDB (Mongoose ODM)
- **Authentication:** JWT, Passport.js, Bcryptjs
- **File Storage:** Cloudinary (Multer storage)
- **Email:** Nodemailer (OTP/Notifications/Password Reset/Invoices)
- **Payments:** Razorpay
- **PDF Generation:** PDFKit

- **Security:** Helmet, Express-Rate-Limit, XSS-Clean, HPP
- **Validation:** Joi

Data Coverage

The application manages the following core data entities:

- **Users:** Authentication credentials, profile details, roles (Admin/User), and organization info.
- **Projects:** Architecture projects with metadata, status, description, and associated media.
- **Designs:** Visual design assets, categories, and gallery images.
- **OTPs:** Temporary verification codes for secure signup/login.

Project Structure

MKPDesigns/

```

├── client/           # Frontend Application
|   ├── src/
|   |   ├── pages/    # Route components (Dashboard, Login, etc.)
|   |   ├── components/ # Reusable UI components
|   |   ├── context/  # Global State (Auth, Theme)
|   |   └── services/  # API integration
|   └── public/       # Static assets
|   └── package.json  # Frontend dependencies
|
└── server/          # Backend API
    ├── models/       # Mongoose Schemas (User, Design, Project)
    ├── routes/       # API Endpoints
    ├── controllers/  # Request Logic
    ├── middleware/   # Auth & upload middleware
    └── package.json  # Backend dependencies
  
```

Installation

1. **Clone the repository**
2. `git clone https://github.com/yourusername/MKPDesigns.git`

`cd MKPDesigns`

3. **Install Server Dependencies**
4. `cd server`

npm install

5. Install Client Dependencies

6. cd ../client

npm install

Environment Setup

Create .env files in both client and server directories.

Server (server/.env):

PORT=5000

MONGO_URI=your_mongodb_connection_string

JWT_SECRET=your_jwt_secret

JWT_EXPIRE=30d

Cloundinary

CLOUDINARY_CLOUD_NAME=your_cloud_name

CLOUDINARY_API_KEY=your_api_key

CLOUDINARY_API_SECRET=your_api_secret

Email (Nodemailer)

EMAIL_SERVICE=gmail

EMAIL_USERNAME=your_email@gmail.com

EMAIL_PASSWORD=your_app_password

Google Auth

GOOGLE_CLIENT_ID=your_google_client_id

GOOGLE_CLIENT_SECRET=your_google_client_secret

CLIENT_URL=http://localhost:5173

Client (client/.env):

VITE_API_URL=http://localhost:5000/api

VITE_GOOGLE_CLIENT_ID=your_google_client_id

Running the Application

1. Start the Server

2. cd server

3. npm run dev

Runs on http://localhost:5000

4. **Start the Client**

5. `cd client`

6. `npm run dev`

Runs on `http://localhost:5173`

API Documentation

Major endpoints available:

- **Auth**
 - `POST /api/auth/signup`: Register new user
 - `POST /api/auth/login`: User login
 - `POST /api/auth/google`: Google OAuth
 - `POST /api/auth/verify-otp`: Verify email OTP
 - `POST /api/auth/forgot-password`: Request password reset (OTP/Link)
 - `POST /api/auth/reset-password`: Set new password
- **Projects**
 - `GET /api/projects`: List all projects
 - `POST /api/projects`: Create project (Admin)
 - `GET /api/projects/:id`: Get project details
- **Designs**
 - `GET /api/designs`: List all designs
 - `POST /api/designs`: Upload design (Admin)
- **User**
 - `GET /api/auth/me`: Get current user profile
 - `PUT /api/auth/google/complete`: Complete profile details

Dashboard Pages

- **Overview**: Summary of account and recent activity.
- **Designs**: Gallery view of uploaded architectural designs.
- **Projects**: List and details of ongoing/completed projects.
- **Upload Design**: Interface for adding new design assets.
- **Upload Project**: Interface for adding new projects.
- **Settings**: User preferences and account management.

Data Sources

- **MongoDB:** Primary persistent storage for all application data.
- **Cloudinary:** Cloud storage for project images and design assets.

Deployment

Frontend (Vercel/Netlify)

1. Build the client: `npm run build`
2. Deploy the dist folder.

Backend (Render/Heroku/Railway)

1. Set up environment variables in the dashboard.
2. Deploy the root/server directory.
3. Ensure npm start runs `node server.js`.

Troubleshooting

- **Database Connection Error:** Check your IP whitelist in MongoDB Atlas and ensure `MONGO_URI` is correct.
- **Image Upload Failures:** Verify Cloudinary credentials in `.env`.
- **CORS Issues:** Ensure `CLIENT_URL` in server `.env` matches your frontend URL.

Contributing

1. Fork the project
2. Create your feature branch (`git checkout -b feature/AmazingFeature`)
3. Commit your changes (`git commit -m 'Add some AmazingFeature'`)
4. Push to the branch (`git push origin feature/AmazingFeature`)
5. Open a Pull Request

License

Distributed under the MIT License. See LICENSE for more information.